

Polimorfismo em Java

AULA 9

PROGRAMAÇÃO ORIENTADA A OBJETOS

Um dos pilares fundamentais da Orientação a Objetos — aprenda como objetos diferentes podem responder à mesma mensagem de maneiras distintas, tornando seu código mais flexível, extensível e elegante.

Objetivos de Aprendizagem

Ao final desta aula, você será capaz de:

1

Compreender o Conceito

Entender o que é polimorfismo e por que ele é um pilar fundamental da Orientação a Objetos.

2

Distinguir os Tipos

Diferenciar polimorfismo de sobrecarga (*overloading*) e sobreposição (*overriding*).

3

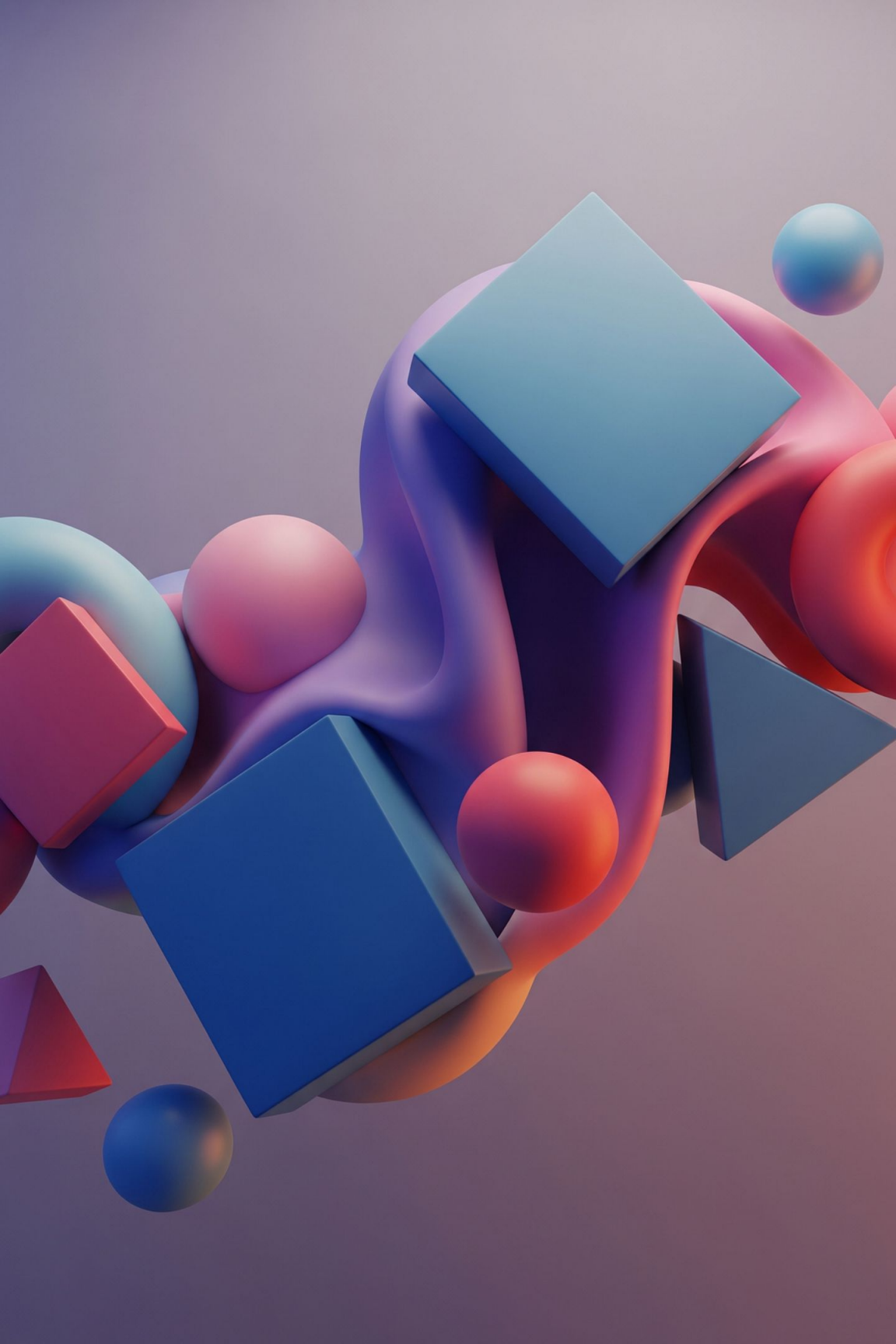
Aplicar na Prática

Usar polimorfismo com herança e interfaces em projetos Java reais.

4

Referenciar Supertipos

Utilizar referências de supertipo para apontar para objetos de subtipo com segurança.



O que é Polimorfismo?

Poli = muitos · **Morphos** = formas

A capacidade de um objeto
assumir *muitas formas*.

Em Java, polimorfismo é a
capacidade de objetos de **tipos
diferentes** responderem à mesma
mensagem (chamada de método) de
maneiras distintas. É o que permite
escrever código genérico que
funciona com qualquer
implementação futura.

Flexibilidade

Código que funciona com tipos
desconhecidos no momento da
escrita.

Extensibilidade

Adicione novos tipos sem
modificar código existente.

Reutilização

Um único método genérico
atende múltiplos tipos.

Tipos de Polimorfismo em Java

1

Sobrecarga

Compile-time Mesmo nome, parâmetros diferentes. Resolvido pelo compilador.

```
void imprimir(int x)
void imprimir(String s)
```

2

Sobreposição

Runtime Reimplementação de método herdado. Resolvido em tempo de execução.

```
@Override
void emitirSom() { ... }
```

3

Herança

Subtipo Referência do tipo pai aponta para objeto filho.

```
Animal a = new Cachorro();
```

4

Interfaces

Contrato Múltiplas implementações de um mesmo contrato.

```
List l = new ArrayList();
```

- ❏ Os dois tipos mais importantes em Java são a **sobreposição (runtime)** e o polimorfismo com **interfaces**, pois ambos permitem ligação dinâmica.

Polimorfismo de Sobreposição (Override)

A sobreposição ocorre quando uma **subclasse reimplementa** um método herdado da superclasse, mantendo a mesma assinatura. A anotação `@Override` é uma boa prática obrigatória.

Mesma Assinatura

Nome, parâmetros e tipo de retorno idênticos ao método pai.

Acesso Igual ou Maior

Não pode restringir a visibilidade do método original.

@Override

Informa ao compilador a intenção — detecta erros de digitação.

```
class Animal {  
    public String toString() {  
        return "Sou um animal";  
    }  
}  
  
class Cachorro extends Animal {  
    @Override  
    public String toString() {  
        return "Sou um cachorro!";  
    }  
}  
  
// Uso:  
Animal a = new Cachorro();  
System.out.println(a.toString());  
// Saída: "Sou um cachorro!"
```

Polimorfismo com Herança

```
Animal a1 = new Cachorro();  
Animal a2 = new Gato();  
Animal a3 = new Passaro();  
  
a1.emitirSom(); // Au au!  
a2.emitirSom(); // Miau!  
a3.emitirSom(); // Piu piu!
```

📌 **Ligação Dinâmica:** o método executado é o da **classe do objeto**, não da referência. O Java decide em tempo de execução.

Como funciona a ligação dinâmica?

Quando você declara `Animal a = new Cachorro();`, a variável `a` tem o tipo de referência **Animal**, mas aponta para um objeto do tipo **Cachorro**.

1 Compilação

O compilador verifica se o método existe em `Animal`.

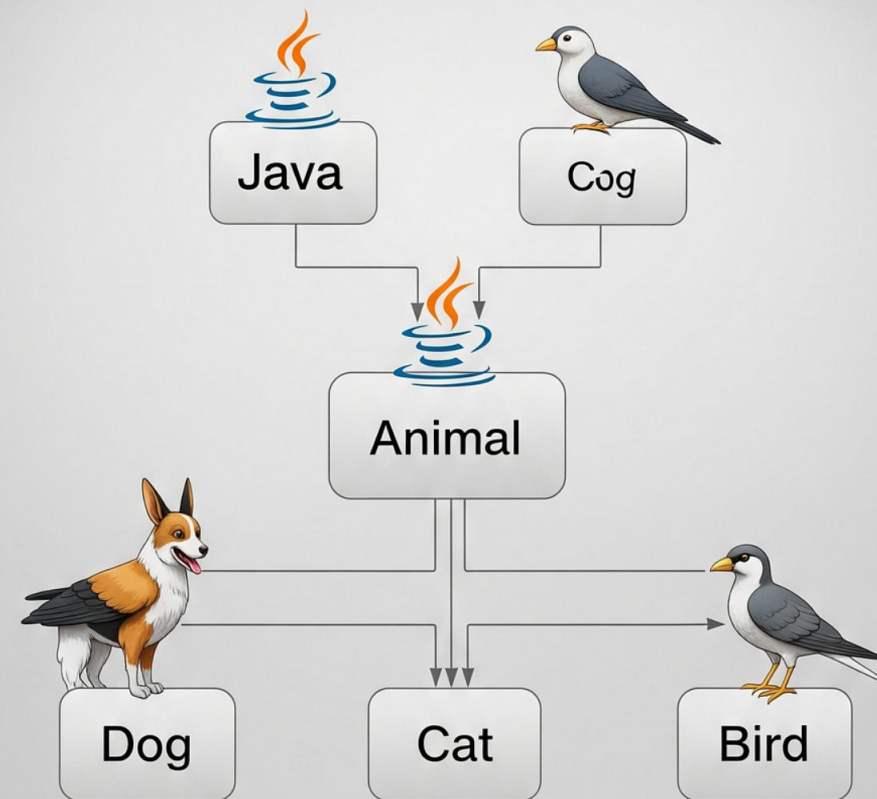
2 Execução

A JVM procura o método na vtable do objeto real (`Cachorro`).

3 Despacho

Executa a versão sobreposta na subclasse.

Exemplo Prático: Hierarquia de Animais



```
class Animal {
    public void emitirSom() {
        System.out.println("...");
    }
}

class Cachorro extends Animal {
    @Override
    public void emitirSom() { System.out.println("Au au!"); }
}

class Gato extends Animal {
    @Override
    public void emitirSom() { System.out.println("Miau!"); }
}

class Passaro extends Animal {
    @Override
    public void emitirSom() { System.out.println("Piu piu!"); }
}

// Polimorfismo em ação com array:
Animal[] animais = { new Cachorro(), new Gato(), new Passaro() };
for (Animal a : animais) {
    a.emitirSom(); // Cada um emite seu próprio som!
}
```

O array de `Animal` funciona com **qualquer** subclasse, presente ou futura, sem modificar o laço.

Regras para Sobreposição

Mesma Assinatura

Nome, lista de parâmetros e tipo de retorno devem ser idênticos ao método da superclasse.



Acesso Não Restritivo

O modificador pode ser igual ou *menos* restritivo. `protected` pode virar `public`, mas nunca `private`.

Não Sobrepõe `final/static`

Métodos `final` são bloqueados para sobreposição.
Métodos `static` pertencem à classe, não ao objeto.



Construtores Excluídos

Construtores **nunca** são sobrepostos — eles pertencem exclusivamente à classe que os define.

Palavras-chave: `super` e `this`

`super` — Acessa o pai

```
class Cachorro extends Animal {  
    @Override  
    public void emitirSom() {  
        super.emitirSom(); // chama Animal  
        System.out.println("Au au!");  
    }  
  
    public Cachorro(String nome) {  
        super(nome); // construtor pai  
    }  
}
```

`super.metodo()` executa a implementação da superclasse antes de adicionar o comportamento da subclasse.

`this` — Referência ao objeto atual

```
class Animal {  
    private String nome;  
  
    public Animal(String nome) {  
        this.nome = nome; // this.campo  
    }  
  
    public Animal(String nome, int idade) {  
        this(nome); // chama outro construtor  
        // ...  
    }  
}
```

`this` desambigua campos da classe de parâmetros locais e permite encadeamento de construtores.

Polimorfismo com Interfaces

Interfaces definem um **contrato** — um conjunto de métodos que qualquer implementação deve fornecer. Isso é a base do princípio *"programa para interfaces, não para implementações"*.

```
interface Pagamento {
    void processar(double valor);
}

class CartaoCredito implements Pagamento {
    @Override
    public void processar(double valor) {
        System.out.println("Processando R$" +
valor + " no cartão...");
    }
}
```

```
class PayPal implements Pagamento {
    @Override
    public void processar(double valor) {
        System.out.println("Enviando R$" + valor
+ " via PayPal...");
    }
}

// Referência de interface → qualquer
implementação:
Pagamento p = new PayPal();
p.processar(150.00); // funciona para qualquer
Pagamento!
```

❏ O mesmo padrão é usado no Java Collections Framework: `List lista = new ArrayList<>();` — troca a implementação sem alterar o código consumidor.

Exemplo Prático: Sistema de Pagamentos

```
interface Pagamento {
    void processar(double valor);
    String getDescricao();
}

class Boleto implements Pagamento {
    @Override
    public void processar(double v)
    {
        System.out.println(
            "Boleto gerado: R$" + v);
    }
    @Override
    public String getDescricao() {
        return "Boleto Bancário";
    }
}
```

```
// Processador genérico:
class Processador {
    void executar(Pagamento p,
        double v) {
        System.out.println("Via: "
            + p.getDescricao());
        p.processar(v);
    }
}

// Na prática:
Processador proc = new
    Processador();
proc.executar(new CartaoCredito(),
    200.0);
proc.executar(new PayPal(), 50.0);
proc.executar(new Boleto(), 300.0);
```

Por que isso é poderoso?

Zero mudanças no Processador

Para adicionar **Pix**, basta criar uma nova classe — o **Processador** não precisa mudar.

Testabilidade

Fácil criar implementações mock para testes unitários.

Princípio Aberto/Fechado

Aberto para extensão, fechado para modificação — um princípio SOLID.

Ligação Estática vs. Dinâmica

Característica	Ligação Estática (Compile-time)	Ligação Dinâmica (Runtime)
Quando ocorre	Na compilação	Na execução (JVM)
Aplicável a	Métodos <code>static</code> , <code>final</code> , <code>private</code>	Métodos de instância sobrepostos
Polimorfismo real	Não — sempre o mesmo método	Sim — método do objeto real
Performance	Ligeiramente mais rápido	Overhead mínimo (vtable)
Exemplo	<code>Math.sqrt()</code>	<code>animal.emitirSom()</code>

Na prática, o overhead da ligação dinâmica é **desprezível** na JVM moderna — não optimize prematuramente evitando métodos virtuais.

Operador `instanceof` e Cast de Referências

Operador `instanceof`

```
Animal a = new Cachorro();

if (a instanceof Cachorro) {
    Cachorro c = (Cachorro) a; // downcast seguro
    c.buscarBola();
}

// Java 16+: pattern matching
if (a instanceof Cachorro c) {
    c.buscarBola(); // sem cast explícito!
}
```

- ❏ Use `instanceof` antes de todo downcast para evitar `ClassCastException` em tempo de execução.

Tipos de Cast

Upcast (implícito)

Filho → Pai. Sempre seguro, feito automaticamente.

```
Animal a = new Cachorro();
```

Downcast (explícito)

Pai → Filho. Requer cast manual e verificação.

```
Cachorro c = (Cachorro) a;
```

ClassCastException

Ocorre quando o objeto real não é do tipo do cast:

```
(Gato) new Cachorro() // ERRO!
```

Boas Práticas



Programa para Interfaces

Declare variáveis com o tipo da interface ou superclasse: `List` em vez de `ArrayList`. Isso desacopla o código da implementação concreta.



Use `@Override` Sempre

A anotação faz o compilador verificar se há realmente uma sobreposição. Sem ela, um erro de digitação cria um novo método silenciosamente.



Prefira Composição à Herança

Para reutilização de código, composição costuma ser mais flexível. Use herança apenas quando há uma relação genuína de "é um".



Evite Downcasts Excessivos

Downcasts frequentes indicam design ruim. Se você precisa de comportamento específico, repense a hierarquia ou use interfaces mais granulares.

Erros Comuns

✗ Confundir Sobrecarga com Sobreposição

Sobrecarga muda os **parâmetros** (compile-time).

Sobreposição mantém a **assinatura** idêntica (runtime).

São mecanismos completamente diferentes.

✗ Omitir @Override

Sem a anotação, um erro de digitação como

`emitirSom()` cria um novo método em vez de sobrepor o original — bug silencioso e difícil de encontrar.

✗ Downcast Sem

`inst = (Cachorro) animal` sem verificar o tipo real pode lançar `ClassCastException` em produção, derrubando a aplicação.

✗ Violar o Princípio de Liskov

Uma subclasse deve ser substituível pela superclasse **sem quebrar o comportamento esperado**. Sobrepor um método mudando sua semântica viola este princípio SOLID.

Exercícios de Fixação

Pratique os conceitos desta aula com os exercícios a seguir:

01

Identificar Tipos de Polimorfismo

Dado um trecho de código Java, classifique cada chamada de método como sobrecarga, sobreposição, polimorfismo com herança ou com interface. Justifique sua resposta.

03

Sistema com Interfaces

Crie a interface `Notificavel` com método `notificar(String msg)`. Implemente `Email`, `SMS` e `PushNotification`. Escreva um serviço genérico que aceite qualquer `Notificavel`.

02

Hierarquia Polimórfica

Crie uma hierarquia `Veiculo` → `Carro`, `Moto`, `Caminhao`. Cada subclasse deve sobrepor `calcularIPVA()` com regra diferente. Use um array de `Veiculo` para calcular o total.

04

Debug com Polimorfismo

Analise o código fornecido pelo professor que contém erros de sobreposição incorreta, downcast inseguro e violação do `@Override`. Identifique e corrija todos os bugs.

📌 **Dica do Professor:** Para cada exercício, desenhe o diagrama UML de classes antes de escrever o código. Visualizar a hierarquia ajuda a identificar onde o polimorfismo será aplicado.